SODA Challenge

Saudi Organic Dates Classification

AI-Powered Classification of Saudi Organic Date Varieties

Team Tamra

Jana Saati	446017177	G5
Nouf bin Taleb	446002883	G6
Bayader Al-fefe	446003924	G6

DS2313 - Data Analysis 1 | Umm Al-Qura University | 2026

Table of Contents

01 **Competition Overview**
SODA Challenge & Dataset

02 **Exploratory Data Analysis**
EDA, Statistics & Insights

03 **Model Journey**
From 0.41 to 0.838 F1-Score

04 **Code Walkthrough**
Data Prep, Model, TTA & Submission

05 **Results & Comparison**
Model Performance & Leaderboard

06 **Conclusion**
Insights & Future Work

01 | Competition Overview

What is SODA Challenge?

- CCWin Challenges Series 1 — hosted on Kaggle
- Classify 13 varieties of Saudi organic dates from images
- Hosted by UQU College of Computing
- Evaluation Metric: Macro F1-Score
- 5 submissions/day | 2 final selections

1,829
Training Images

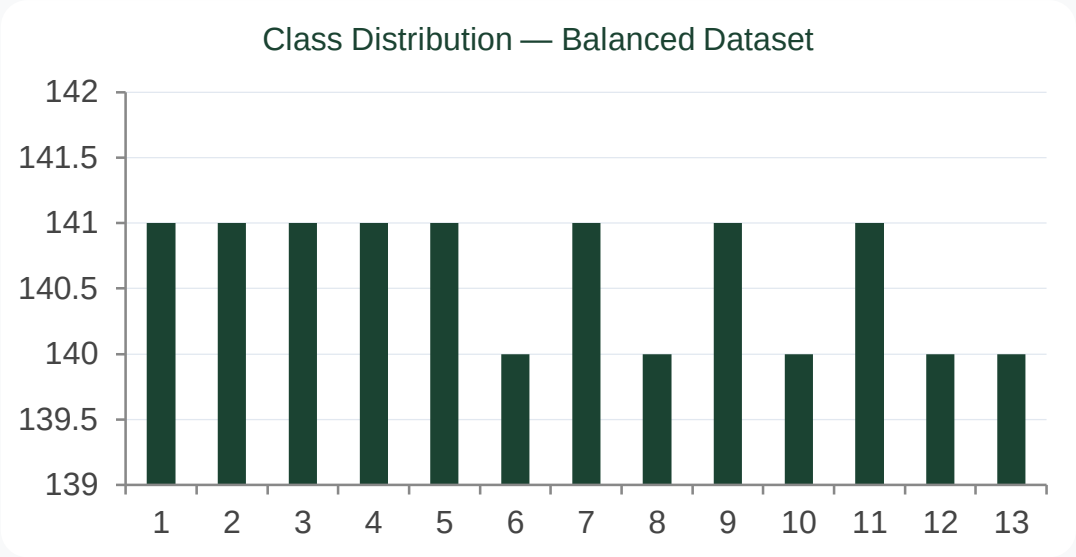
784
Test Images

13
Date Varieties

~141
Images/Class

Submission Format
CSV file with columns: id, label
Max 5 submissions/day | 2 final selections

02 | Exploratory Data Analysis (EDA)



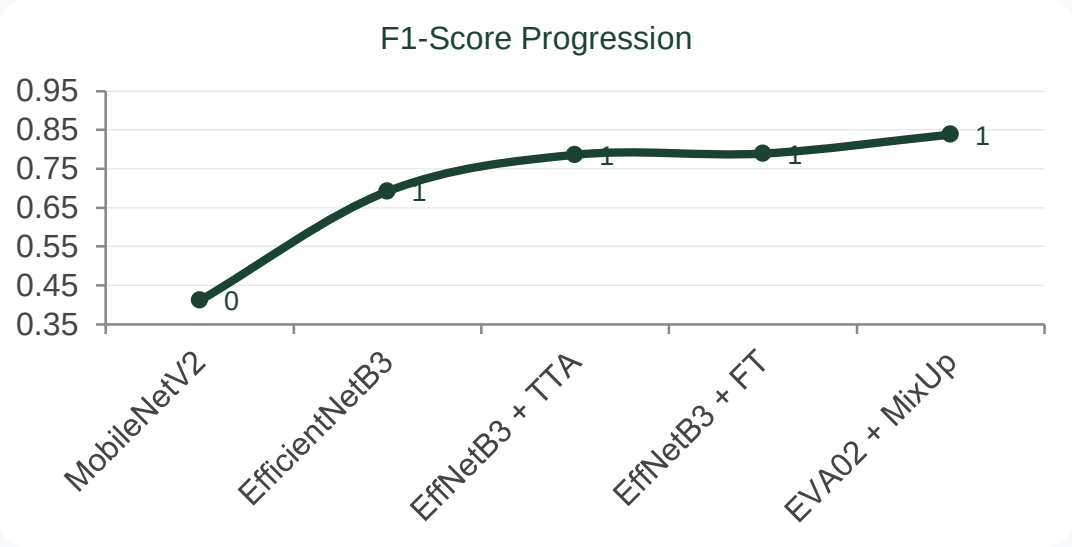
Dataset Summary

Total Images:	2,613 (train: 1,829 | test: 784)
Formats:	Mixed JPG & PNG
Classes:	13 balanced varieties (140–141 each)

Key Findings

- Perfectly balanced classes — equal representation
- Mixed image formats: JPG & PNG
- 13 distinct Saudi date varieties
- ~140-141 images per class
- Macro F1 chosen — ideal for balanced data
- No missing values or corrupt files found

03 | Model Journey



MobileNetV2
Baseline | Basic Augmentation
0.411

EfficientNetB3
Better Backbone | More Epochs
0.692

EffNetB3 + TTA
Test Time Augmentation | 5×
0.786

EffNetB3 + FT
Fine-Tuning | 150 Frozen Layers
0.789

EVA02 + MixUp
PyTorch + MixUp & CutMix
0.838

🏆 Final Competition Results (Private Leaderboard)

Team Tamra: Rank #8 | Score: 0.838 F1 | 38 submissions total
1st Place: Data Minds | 0.891 F1 | Gap: ~0.053

04 | Code: Data Preparation

```
# 1. Load & prepare data
train = pd.read_csv('train.csv')
train['label'] = train['label'].astype(str)

# 2. Find image with correct extension
def get_filename(img_id, path):
    for ext in ['.jpg', '.png', '.jpeg']:
        if os.path.exists(path + ext):
            return img_id + ext

# 3. Compute balanced class weights
class_weights = compute_class_weight(
    'balanced', classes, y=train['label'])

# 4. Stratified Train / Validation split
train_df, val_df = train_test_split(
    train, test_size=0.15,
    stratify=train['label'], random_state=42)
```

1

Load Data

Read train.csv & test.csv, convert labels to string format for Keras generators

2

Find Images

Function to locate each image file with its correct extension (JPG or PNG)

3

Class Weights

Compute balanced weights — ensures the model treats all 13 classes equally

4

Train/Val Split

85% training / 15% validation, stratified to maintain class balance in both sets

04 | Code: TTA & Submission File

```
# Test Time Augmentation (10 rounds)
tta_preds = np.zeros((len(test), 13))

for i in range(10):
    tta_gen = ImageDataGenerator(
        rotation_range=20,
        horizontal_flip=True,
        zoom_range=0.1)
    preds = model.predict(tta_gen)
    tta_preds += preds  # accumulate

final = np.argmax(tta_preds, axis=1)
```

```
# Generate submission CSV file

submission = pd.DataFrame({

    'id': test['id'], 'label': final})

submission.to_csv('submission.csv', index=False)
```

What is TTA?

Test Time Augmentation applies different random transformations to each test image and averages all predictions.

This reduces variance and improves final accuracy significantly.

Without TTA: **~0.76**
With TTA (10×): **0.786** ✓

Submission Format

CSV file with two columns:
- id — test image ID
- label — predicted class (1–13)

04 | Code: Model Architecture & Training

```
# Stage 1: Transfer Learning (freeze backbone)
base = EfficientNetB3(weights='imagenet')
for layer in base.layers[:150]:
    layer.trainable = False
model.compile(optimizer=Adam(lr=1e-4))
model.fit(train_gen, epochs=20, ...)
```

```
# Stage 2: Fine-Tuning (unfreeze all layers)
for layer in base.layers:
    layer.trainable = True
model.compile(optimizer=Adam(lr=1e-5))
model.fit(train_gen, epochs=15, ...)
model = load_model('best_model.keras')
```

Stage 1: Transfer Learning

- Freeze first 150 backbone layers
- Train only the new classification head
- Learning rate: 1e-4 (Adam)
- EarlyStopping + ReduceLROnPlateau
- ModelCheckpoint saves best weights

Stage 2: Fine-Tuning

- Unfreeze all backbone layers
- Very low learning rate: 1e-5
- Prevents catastrophic forgetting
- Auto-save best model checkpoint
- Significantly boosts accuracy

05 | Key Techniques Used

Transfer Learning

Used pre-trained ImageNet weights for MobileNetV2, EfficientNetB3, and EVA02 as starting backbone.

Fine-Tuning

Gradually unfreezing backbone layers and retraining with very low LR (1e-5) to adapt to date images.

Data Augmentation

Rotation, flip, zoom, brightness, shear, MixUp, and CutMix to improve model generalization.

Test Time Augmentation (TTA)

Predicted each test image 5–10 times with different augmentations and averaged for better accuracy.

K-Fold Cross Validation

StratifiedKFold (3–5 folds) ensures robust model evaluation and reduces overfitting.

Class Weights & Label Smoothing

Balanced class weights and label smoothing (0.05–0.1) regularize the model and improve robustness.

06 | Results & Model Comparison

Model	Framework	Key Technique	F1-Score	Outcome
MobileNetV2	TensorFlow	Baseline only	0.411	✕ Weak
EfficientNetB3	TensorFlow	Better backbone	0.692	◈ Improved
EfficientNetB3 + TTA	TensorFlow	5× TTA	0.786	◈ Good
EfficientNetB3 + FT	TensorFlow	Full Fine-Tuning	**0.789**	✓✓ Best TF
EVA02 + MixUp/CutMix	PyTorch	MixUp + CutMix + KFold	**0.838**	🏆 Final Best

🎖 Final Rank: #8 | Score: 0.838 F1
38 total submissions

🥇 1st Place: Data Minds | 0.891 F1
Gap to 1st: ~0.053

07 | Conclusion

1. Transfer Learning with EfficientNetB3 and EVA02 significantly outperformed baseline models

2. Data Augmentation (MixUp, CutMix, rotation, flip) was essential for generalization

3. Test Time Augmentation (TTA) consistently improved prediction robustness

4. K-Fold Cross Validation reduced overfitting and improved model reliability

Future Work

- → Ensemble multiple models
- → Use external date datasets
- → Higher resolution (512px+)
- → EfficientNetV2L backbone
- → Longer training schedule

Final Score: 0.838 F1 | Rank #8 / Private LB | Team Tamra | DS2313 - UQU 2026